

**ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



**BÁO CÁO
ĐỒ ÁN TỐT NGHIỆP**

**XÂY DỰNG HỆ THỐNG QUẢN LÝ THƯ VIỆN
TRỰC TUYẾN TÍCH HỢP AI HỖ TRỢ
TÌM KIẾM VÀ GỢI Ý SÁCH**

Ngành: Khoa học Máy tính

MÃ ĐỀ TÀI: HK253-DATN-076

HỘI ĐỒNG: 5L Khoa học Máy tính

GVHD: TS. Trương Tuấn Anh

CN. Dương Huỳnh Anh Đức

GVPB: ThS. Trần Thị Quê Nguyệt

–o0o–

SVTH: Võ Quang Thắng - 2213214

TP. HỒ CHÍ MINH, 09/2026

LỜI CAM ĐOAN

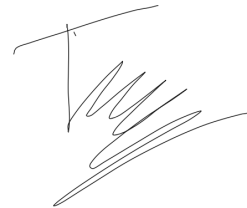
Em xin cam đoan đồ án tốt nghiệp với đề tài **“Xây dựng hệ thống quản lý thư viện trực tuyến tích hợp AI hỗ trợ tìm kiếm và gợi ý sách”** là công trình do em thực hiện dưới sự hướng dẫn của **TS. Trương Tuấn Anh** và **CN. Dương Huỳnh Anh Đức**.

Các nội dung nghiên cứu, thiết kế, triển khai và đánh giá trong báo cáo được trình bày trung thực. Những tài liệu, số liệu, hình ảnh, công cụ và kết quả tham khảo từ các nguồn khác đều được trích dẫn hoặc ghi nhận trong phần tài liệu tham khảo và các phần liên quan của báo cáo.

Em xin chịu trách nhiệm trước Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách Khoa - Đại học Quốc gia TP.HCM về tính trung thực và chính xác của nội dung báo cáo này.

Tp. Hồ Chí Minh, tháng 09 năm 2026

Sinh viên thực hiện



Võ Quang Thắng

LỜI CẢM ƠN

Trong suốt thời gian học tập và nghiên cứu tại **Trường Đại học Bách Khoa - Đại học Quốc gia Thành phố Hồ Chí Minh**, em đã nhận được sự quan tâm, giảng dạy và hướng dẫn tận tình từ quý thầy cô trong khoa. Đồ án tốt nghiệp này là minh chứng cho sự nỗ lực không ngừng của em trong việc vận dụng những kiến thức đã được trang bị để giải quyết các vấn đề thực tiễn trong lĩnh vực công nghệ thông tin.

Trước tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến **TS. Trương Tuấn Anh**, người thầy đã tận tâm hướng dẫn, định hướng và đồng hành cùng em trong suốt quá trình thực hiện đồ án. Thầy không chỉ truyền đạt kiến thức chuyên môn mà còn định hướng phương pháp nghiên cứu khoa học, giúp em có cái nhìn sâu sắc hơn về vấn đề đang giải quyết. Những lời khuyên, góp ý quý báu của thầy là động lực to lớn giúp em vượt qua mọi khó khăn, thử thách để hoàn thành tốt đồ án này.

Em cũng xin gửi lời cảm ơn chân thành đến **CN. Dương Huỳnh Anh Đức** đã đồng hành, hỗ trợ và đóng góp nhiều ý kiến quý báu trong quá trình thực hiện đồ án, giúp em có thêm góc nhìn thực tiễn và áp dụng hiệu quả các giải pháp kỹ thuật.

Em xin bày tỏ lòng biết ơn sâu sắc đến toàn thể quý thầy cô **Khoa Khoa học và Kỹ thuật Máy tính** đã tận tình giảng dạy, truyền đạt kiến thức và kinh nghiệm quý báu trong suốt những năm học vừa qua. Nền tảng kiến thức vững chắc mà các thầy cô đã trang bị là hành trang vô cùng quý giá để em tự tin bước vào môi trường làm việc thực tế.

Em cũng xin cảm ơn gia đình, bạn bè đã luôn bên cạnh, động viên và tạo điều kiện tốt nhất để em có thể hoàn thành tốt chương trình học tập cũng như đồ án tốt nghiệp này.

Do thời gian và kinh nghiệm có hạn, đồ án không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý, chỉ bảo của quý thầy cô để đồ án được hoàn thiện hơn và em có thêm kinh nghiệm quý báu cho công việc sau này.

Xin chân thành cảm ơn!

Tp. Hồ Chí Minh, tháng 09 năm 2026

Sinh viên thực hiện



Võ Quang Thắng

TÓM TẮT

Đồ án tốt nghiệp này tập trung vào việc nghiên cứu, thiết kế và xây dựng **Hệ thống Quản lý Thư viện Trực tuyến tích hợp AI**, nhằm giải quyết các hạn chế của các hệ thống thư viện truyền thống như khả năng tìm kiếm kém hiệu quả và thiếu sự cá nhân hóa. Hệ thống được phát triển với mục tiêu số hóa toàn diện quy trình vận hành thư viện, đồng thời ứng dụng trí tuệ nhân tạo để nâng cao trải nghiệm đọc và khám phá tri thức của người dùng.

Về mặt kỹ thuật, hệ thống được xây dựng dựa trên kiến trúc **Modular Monolith** với Clean Architecture + Domain-Driven Design cho từng module, đảm bảo tính bảo mật và khả năng mở rộng. Backend sử dụng **Spring Boot (Java)** để xử lý các nghiệp vụ phức tạp, kết hợp với Frontend **React + Vite** mang lại giao diện hiện đại, tốc độ build nhanh và triển khai đơn giản qua NGINX. Các chức năng chính bao gồm: quản lý tài nguyên sách, quản lý độc giả, quy trình mượn trả tự động, kiểm soát phí phạt và hệ thống báo cáo thống kê trực quan cho thủ thư.

Điểm đột phá của đồ án là việc tích hợp sâu các công nghệ AI/NLP tiên tiến. Cụ thể, hệ thống ứng dụng mô hình **Sentence-BERT (SBERT)** để thực hiện tìm kiếm ngữ nghĩa (Semantic Search), giúp xử lý các truy vấn tiếng Việt phức tạp mà không phụ thuộc vào từ khóa chính xác. Song song đó, hệ thống gợi ý sách sử dụng phương pháp lai (**Hybrid Approach**), kết hợp giữa Lọc cộng tác (Collaborative Filtering) và Lọc dựa trên nội dung (Content-Based), giúp đưa ra các đề xuất sách phù hợp nhất dựa trên lịch sử và hành vi của từng độc giả.

Kết quả của đồ án là một hệ thống phần mềm hoàn chỉnh, có giao diện thân thiện, đáp ứng đầy đủ các yêu cầu nghiệp vụ của một thư viện đại học. Hệ thống đã chứng minh được tính khả thi trong việc kết hợp giữa quản lý truyền thống và công nghệ thông minh, mang lại giá trị thực tiễn cao trong việc thúc đẩy văn hóa đọc và chuyển đổi số trong môi trường giáo dục.

Mục lục

Lời cam đoan	i
Lời cảm ơn	ii
Tóm tắt	iii
Danh mục hình ảnh	vi
Danh mục bảng biểu	vii
1 Triển khai hệ thống	1
1.1 Caddy — Reverse Proxy	1
1.1.1 Vai trò của Caddy	1
1.2 Môi trường sản xuất	1
1.2.1 Docker Compose sản xuất	1
1.2.2 Quản lý biến môi trường và bảo mật	2
1.2.3 Flyway Migration tự động	2
1.3 Quy trình khởi tạo hạ tầng ban đầu	3
1.4 Hệ thống Tích hợp và Triển khai liên tục (CI/CD Pipeline)	4
1.4.1 Đặt vấn đề và mục tiêu thiết kế	4
1.4.2 Kiến trúc tổng thể và luồng hoạt động	5
1.4.3 Chi tiết giai đoạn Tích hợp liên tục (CI)	6
1.4.4 Chi tiết giai đoạn Triển khai liên tục (CD)	7
1.4.5 Quản lý bảo mật và phân tách cấu hình bí mật	9
1.4.6 Các thách thức kỹ thuật và giải pháp xử lý thực tế	9
1.4.7 Đánh giá định lượng hiệu quả hệ thống	10
1.5 Hạ tầng mạng: Domain và HTTPS	11
1.5.1 Tên miền (Domain)	11
1.5.2 Tổng chi phí vận hành	11
2 Tổng kết	13
2.1 Tổng hợp kết quả hiện thực	13
2.1.1 Phân lớp Frontend	13
2.1.2 Phân lớp Backend	14
2.1.3 Phân lớp AI Service	14
2.1.4 Kiểm thử và Đảm bảo chất lượng	15

2.1.5	Đóng gói và Triển khai hạ tầng	15
2.2	Đánh giá tác động thực tiễn	16
2.3	Nhận định kiến trúc và giới hạn kỹ thuật	16
2.4	Định hướng phát triển	17
2.5	Kết luận chung	17

Tài liệu tham khảo

Tài nguyên và Mã nguồn dự án

Danh sách hình vẽ

1.1	Kiến trúc tổng thể luồng CI/CD trong hệ thống Library74	6
1.2	Luồng kết nối từ tên miền Library74 đến hệ thống production	11
2.1	Nhận diện thương hiệu của nền tảng Library74	13

Danh sách bảng

1.1	Các nhóm biến môi trường của hệ thống	2
1.2	Chi tiết các workflow Tích hợp liên tục (CI)	7
1.3	Đánh giá hiệu quả vận hành trước và sau khi áp dụng CI/CD	10
1.4	Chi phí vận hành hệ thống trong cấu hình demo	12
1	Danh sách các tài nguyên trực tuyến của hệ thống	

Chương 1: Triển khai hệ thống

1.1 Caddy — Reverse Proxy

1.1.1 Vai trò của Caddy

Caddy đóng vai trò là reverse proxy trung tâm, tiếp nhận toàn bộ traffic public từ bên ngoài trên cổng 80/443, tự động cấp HTTPS và định tuyến request đến đúng dịch vụ nội bộ. Kiến trúc này mang lại hai lợi ích chính: thứ nhất, các cổng nội bộ của từng container không bị lộ trực tiếp ra internet; thứ hai, client chỉ cần giao tiếp với một địa chỉ duy nhất là `https://library74.uk`. Trong mô hình hiện tại, Nginx vẫn được sử dụng bên trong container frontend để phục vụ static build của React/Vite, còn Caddy là lớp endpoint và reverse proxy ở bên ngoài.

1.2 Môi trường sản xuất

1.2.1 Docker Compose sản xuất

Trong repository hiện tại, cấu hình container được tách theo từng thành phần: `LMS_BE/docker-compose.yml` quản lý PostgreSQL và Kafka cho backend, còn `LMS_AI/docker-compose.yml` quản lý RabbitMQ, FastAPI API server và Celery worker. Frontend và backend có Dockerfile riêng để build image triển khai. Khi đưa lên server, các thành phần này cần được đặt trong cùng một mạng triển khai hoặc được cấu hình endpoint rõ ràng thông qua biến môi trường.

Luồng kết nối quan trọng nhất là backend gọi AI qua `AI_GATEWAY_BASE_URL`, còn AI service gọi ngược backend qua `BACKEND_WEBHOOK_URL`. Secret ký callback của hai phía phải khớp nhau: `AI_CALLBACK_SECRET` ở backend và `BACKEND_WEBHOOK_SECRET` ở AI service.

Một điểm khác biệt quan trọng so với cấu hình phát triển là giá trị `KAFKA_CFG_ADVERTISED_LISTENERS` được thay đổi từ `PLAINTEXT://localhost:9092` thành `PLAINTEXT://kafka:9092`. Trong môi trường phát triển, Spring Boot chạy trên máy host nên dùng `localhost`; trong môi trường container, Spring Boot nằm trong một container khác và phải kết nối đến Kafka qua tên service Docker.

```
# Hạ tầng Backend
cd LMS_BE && docker compose up -d

# AI Service
cd ../LMS_AI && docker compose up -d --build
```

```
# Build image Backend và Frontend
cd ../LMS_BE && docker build -t lms-backend .
cd ../LMS_fe && docker build -t lms-frontend .
```

1.2.2 Quản lý biến môi trường và bảo mật

Tất cả thông tin nhạy cảm của hệ thống được quản lý thông qua file `.env` và không bao giờ được đưa vào mã nguồn (được liệt kê trong `.gitignore`). Bảng 1.1 liệt kê các nhóm biến môi trường chính.

Bảng 1.1: Các nhóm biến môi trường của hệ thống

Nhóm	Biến	Mục đích
Cơ sở dữ liệu	DATABASE_URL DATABASE_USERNAME DATABASE_PASSWORD	Kết nối PostgreSQL
JWT	EXP_TOKEN EXP_REFRESH_TOKEN	Thời hạn Access/Refresh token
OAuth2 Google	GOOGLE_CLIENT_ID GOOGLE_CLIENT_SECRET OAUTH2_REDIRECT_URI	Đăng nhập Google
Email	MAIL_SENDER_USERNAME MAIL_SENDER_PASSWORD	Gửi email qua SMTP
AWS S3	AWS_ACCESS_KEY AWS_SECRET_KEY	Lưu trữ file (ảnh bìa, tài liệu)
AI Gateway	AI_GATEWAY_BASE_URL AI_CALLBACK_SECRET AI_CALLBACK_MAX_CLOCK_ SKEW_SECONDS	Backend gọi AI service và xác thực callback
AI Service	GEMINI_API_KEY RABBITMQ_URL BACKEND_WEBHOOK_URL BACKEND_WEBHOOK_SECRET	Chạy FastAPI, Celery, Gemini và webhook
PayOS	CLIENT_ID_PAYOS API_KEY_PAYOS CHECKSUM_KEY_PAYOS	Tạo và xác thực thanh toán phí phạt
Frontend	VITE_API_BASE_URL VITE_API_TIMEOUT	Cấu hình endpoint API và timeout khi build Vite

Khi triển khai lên máy chủ, mỗi thành phần có file môi trường riêng: `LMS_BE.env`, `LMS-AI.env` và cấu hình build cho `LMS_fe`. Các file này được tạo từ `.env.example`, điền giá trị thực tế trên server và không đưa vào lịch sử git. Với Docker Compose, các biến được tham chiếu qua `env_file` hoặc truyền vào build argument để tránh nhúng secret vào Docker image.

1.2.3 Flyway Migration tự động

Hệ thống sử dụng Flyway để quản lý phiên bản schema cơ sở dữ liệu. Khi container Spring Boot khởi động, Flyway tự động thực thi các file migration chưa được áp dụng theo thứ tự phiên bản trước khi ứng dụng nhận request. Điều này đảm bảo schema cơ sở dữ liệu luôn đồng bộ với

phiên bản ứng dụng mà không cần can thiệp thủ công.

```
library-bootstrap/src/main/resources/db/migration/  
+-- V1__create_schema.sql  
+-- V8__ai_publication_etl_status.sql  
+-- V14__metadata_translations.sql  
+-- V20__schema_cleanup_and_ai_indexes.sql  
+-- V22__cleanup_ai_metadata_junk.sql
```

1.3 Quy trình khởi tạo hạ tầng ban đầu

Quy trình thiết lập hệ thống từ đầu trên một máy chủ Linux (Ubuntu Server) trắng bao gồm các bước sau:

1. **Chuẩn bị môi trường máy chủ:** Cài đặt Docker Engine và Docker Compose Plugin chính thức từ Docker repository:

```
curl -fsSL https://get.docker.com | sh  
sudo usermod -aG docker $USER
```

2. **Sao chép mã nguồn:** Tải repository dự án về thư mục quản trị trên máy chủ:

```
git clone <repository-url> /opt/library74  
cd /opt/library74
```

3. **Cấu hình môi trường vận hành:** Khởi tạo các file biến môi trường bảo mật từ mẫu cấu hình và thiết lập giá trị thực tế (thông số kết nối database, khóa bí mật JWT, API key của bên thứ ba):

```
cp LMS_BE/.env.example LMS_BE/.env  
cp LMS-AI/.env.example LMS-AI/.env  
cp LMS_fe/.env.example LMS_fe/.env  
chmod 600 LMS_BE/.env LMS-AI/.env LMS_fe/.env
```

4. **Khởi động hệ thống ban đầu:** Đóng gói và kích hoạt toàn bộ các cụm container dịch vụ:

```
cd LMS_BE && docker compose up -d  
cd ../LMS-AI && docker compose up -d --build
```

5. **Kiểm tra trạng thái sẵn sàng:** Xác nhận tất cả container vận hành ổn định và đọc log khởi động ban đầu:

```
docker compose ps  
docker logs -f lms_backend  
docker logs -f lms_ai_api
```

Sau khi khởi động lần đầu, Flyway tự động thực thi các tệp migration của backend. Cụm AI service tự động đồng bộ lược đồ vector, kết nối Message Broker RabbitMQ, tải nạp mô hình Sentence-Transformers về bộ nhớ cache cục bộ và sẵn sàng phục vụ các truy vấn nghiệp vụ.

1.4 Hệ thống Tích hợp và Triển khai liên tục (CI/CD Pipeline)

1.4.1 Đặt vấn đề và mục tiêu thiết kế

Trong các dự án phần mềm quy mô lớn kết hợp đa nền tảng công nghệ (Java/Spring Boot, Python/FastAPI, React/TypeScript), việc áp dụng quy trình triển khai thủ công (Manual Deployment) bộc lộ nhiều điểm nghẽn nghiêm trọng:

- **Rủi ro sai sót thao tác (Human Error):** Lập trình viên phải trực tiếp truy cập SSH vào máy chủ sản xuất, tự thực hiện các lệnh kéo mã nguồn và build container, rất dễ nhầm lẫn biến môi trường hoặc xung đột phiên bản.
- **Mã nguồn lỗi lọt vào môi trường Production:** Nếu không có rào chắn kiểm thử tự động bắt buộc trước khi đóng gói, các commit chứa lỗi logic hoặc lỗi hồi quy (regression) có thể làm tê liệt dịch vụ ngay khi đưa lên server.
- **Rủi ro thất thoát dữ liệu:** Quá trình cập nhật phiên bản backend mới thường đi kèm các thay đổi về cấu trúc cơ sở dữ liệu (Flyway migration). Nếu không có cơ chế tự động sao lưu dữ liệu tức thì (snapshot) ngay trước thời điểm deploy, việc khôi phục dữ liệu khi xảy ra lỗi là vô cùng phức tạp.
- **Thời gian ngưng trệ dịch vụ (Downtime) kéo dài:** Việc biên dịch ứng dụng và build Docker image trực tiếp trên máy chủ gây quá tải CPU/RAM, ảnh hưởng trực tiếp đến người dùng đang hoạt động.

Nhằm khắc phục toàn diện các vấn đề trên, hệ thống CI/CD của dự án Library74 được thiết kế dựa trên nền tảng GitHub Actions với 4 mục tiêu cốt lõi:

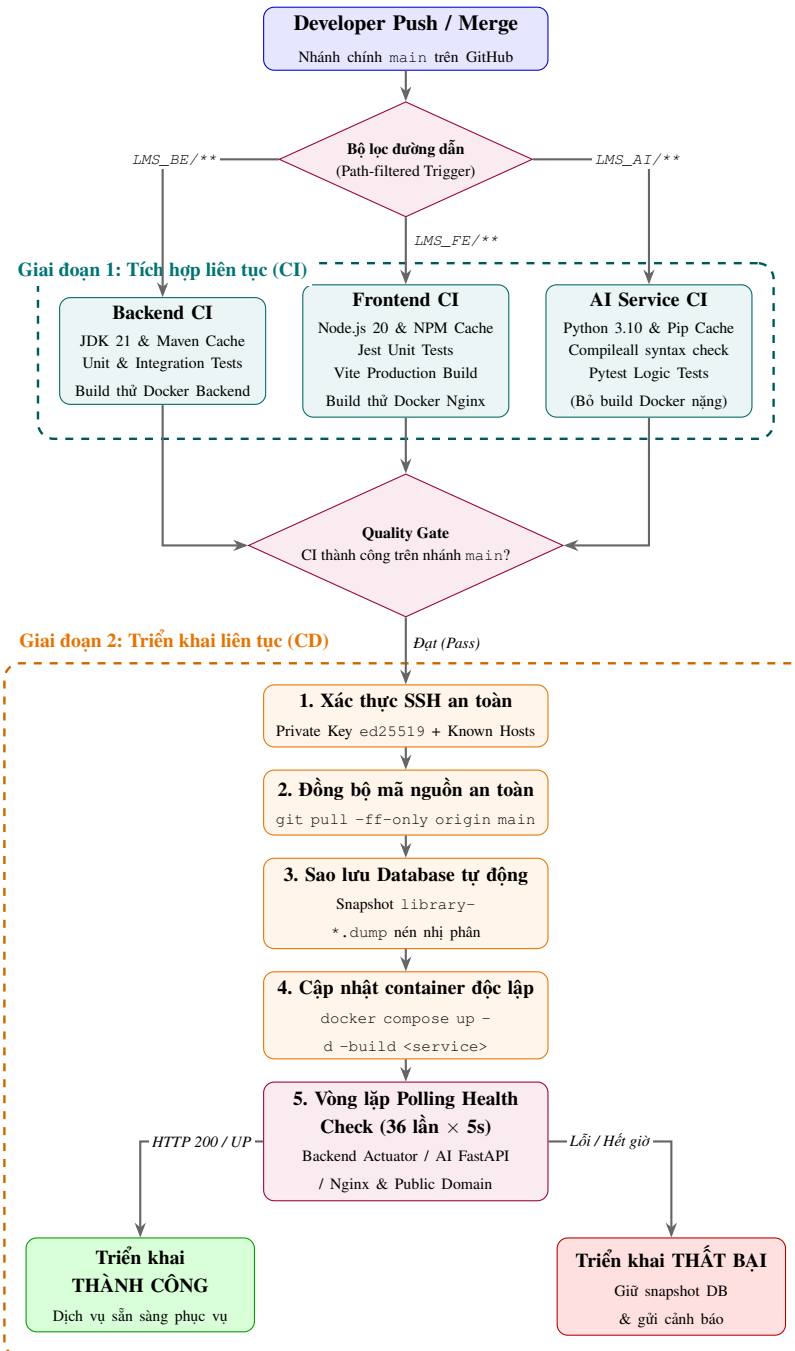
1. **Tự động hóa toàn diện 100% (End-to-End Automation):** Mọi thay đổi mã nguồn sau khi được gộp (merge) vào nhánh chính `main` đều tự động kích hoạt kiểm thử, đóng gói và triển khai lên máy chủ mà không cần can thiệp thủ công.
2. **Kiến trúc Monorepo phân tách theo module (Path-filtered Pipeline):** Bộ lọc đường dẫn thay đổi giúp tách biệt hoàn toàn pipeline của ba phân hệ độc lập (`LMS_BE`, `LMS_FE`, `LMS_AI`), giúp tiết kiệm tài nguyên và rút ngắn tối đa thời gian phát hành.
3. **An toàn dữ liệu tuyệt đối (Zero Data Loss):** Luôn tự động chụp bản sao lưu cơ sở dữ liệu nhị phân (Database Snapshot) trước khi bất kỳ phiên bản backend mới nào được khởi chạy.
4. **Kiểm tra sức khỏe dịch vụ tự động (Post-deployment Health Check):** Tự động thăm dò trạng thái sẵn sàng của container trước khi chính thức đánh dấu quá trình triển khai thành công.

1.4.2 Kiến trúc tổng thể và luồng hoạt động

Kiến trúc luồng hoạt động của hệ thống CI/CD được phân chia thành bốn giai đoạn kế tiếp nhau với sự phân định rạch ròi về môi trường thực thi:

1. **Giai đoạn Kích hoạt (Trigger):** Lập trình viên đẩy commit hoặc hoàn tất Pull Request sáp nhập vào nhánh `main`. Bộ lọc đường dẫn (*Path Filter*) phân tích các tệp tin sửa đổi để xác định chính xác phân hệ bị ảnh hưởng (`LMS_BE`, `LMS_FE`, hoặc `LMS_AI`).
2. **Giai đoạn Tích hợp liên tục (CI):** Khởi chạy máy ảo độc lập trên GitHub Actions Runner. Từng phân hệ thực hiện khôi phục bộ nhớ đệm (cache), chạy kiểm thử tự động và biên dịch thử nghiệm.
3. **Cổng kiểm soát chất lượng (Quality Gate):** Toàn bộ các bài kiểm tra phải đạt trạng thái `success`. Nếu có bất kỳ kiểm thử nào thất bại, pipeline lập tức dừng lại, gửi thông báo cảnh báo và từ chối tiến hành triển khai.
4. **Giai đoạn Triển khai liên tục (CD):** Máy ảo GitHub kết nối tới máy chủ sản xuất (Production VPS) thông qua giao thức SSH với khóa bảo mật riêng biệt, đồng bộ mã nguồn, thực hiện snapshot dữ liệu, biên dịch container và kiểm tra phản hồi của dịch vụ.

Toàn bộ quy trình chuyển giao tự động và các bước kiểm soát được minh họa chi tiết trong Hình 1.1.



Hình 1.1: Kiến trúc tổng thể luồng CI/CD trong hệ thống Library74

1.4.3 Chi tiết giai đoạn Tích hợp liên tục (CI)

Giai đoạn CI được thực thi trên môi trường máy ảo Ubuntu sạch của GitHub (*GitHub-hosted Runner*). Bảng 1.2 tổng hợp các workflow CI tương ứng với từng thành phần kiến trúc.

Chiến lược tối ưu hóa đặc thù cho phân hệ AI CI: Trong quá trình thiết kế pipeline, phân hệ AI Service sử dụng các thư viện tính toán khoa học và học sâu rất nặng như `torch` (~1GB), `sentence-transformers`, `scipy`. Nếu áp dụng bước `docker build` trên *GitHub-hosted Runner*, máy ảo sẽ phải tải lại toàn bộ PyTorch và model weights từ đầu mà không có lớp cache cục bộ, dẫn đến thời gian thực thi CI kéo dài từ 12 đến 15 phút, gây lãng phí nghiêm trọng quota tính giờ của GitHub Actions.

Bảng 1.2: Chi tiết các workflow Tích hợp liên tục (CI)

Thành phần	Tệp cấu hình	Nhiệm vụ chính	Thời gian
Backend CI	backend-ci.yml .github/workflows/	- Cấu hình OpenJDK 21 Temurin kèm cache Maven. - Thực thi toàn bộ Unit & Integration Test suite. - Build thử nghiệm Docker Image Backend nhằm xác thực Dockerfile multi-stage.	~1–2 phút
Frontend CI	frontend-ci.yml .github/workflows/	- Cấu hình Node.js 20 kèm cache NPM package. - Cài đặt sạch các gói phụ thuộc (npm ci). - Chạy toàn bộ Unit Test giao diện bằng Jest. - Biên dịch gói tĩnh Vite (npm run build). - Build thử nghiệm Docker Image Nginx đóng gói web.	~1–2 phút
AI Service CI	ai-ci.yml .github/workflows/	- Cấu hình Python 3.10 kèm cache Pip wheel. - Biên dịch kiểm tra toàn bộ cú pháp (compileall). - Thực thi bộ test logic và hợp đồng bằng Pytest.	~1–2 phút

Để tối ưu, nhóm phát triển đã đưa ra quyết định kiến trúc: *Tách biệt hoàn toàn việc kiểm thử logic và đóng gói container đối với AI Service*. Phân hệ AI CI trên GitHub Runner chỉ tập trung biên dịch cú pháp mã nguồn và chạy Pytest với môi trường Python chuẩn. Việc đóng gói Docker image được chuyển giao hoàn toàn cho máy chủ VPS trong giai đoạn CD. Tại VPS, Docker Daemon đã lưu trữ sẵn các layer cơ sở của PyTorch và thư viện AI trong bộ nhớ đệm cục bộ, do đó quá trình build lại image khi có cập nhật mã nguồn mới chỉ tiêu tốn từ 3 đến 5 giây.

1.4.4 Chi tiết giai đoạn Triển khai liên tục (CD)

Giai đoạn Triển khai liên tục (CD) được kích hoạt tự động theo cơ chế liên kết sự kiện `workflow_run`, với điều kiện bắt buộc là workflow CI tương ứng đã hoàn thành thành công (`conclusion == 'success'`) trên nhánh `main`, hoặc có thể được người quản trị kích hoạt thủ công thông qua sự kiện `workflow_dispatch`.

Quá trình CD trên máy chủ Production tuân thủ nghiêm ngặt quy trình an toàn 6 bước sau:

- 1. Xác thực bảo mật SSH không mật khẩu (Zero Password Authentication):** Pipeline sử dụng Private Key chuẩn mã hóa hiện đại `ed25519` được lưu trữ an toàn trong GitHub Secrets (`DEPLOY_SSH_KEY`). Trước khi mở kết nối, GitHub Runner tiến hành đối soát dấu vân tay máy chủ thông qua biến `DEPLOY_KNOWN_HOSTS`, loại trừ hoàn toàn nguy cơ bị tấn công giả mạo máy chủ (Man-in-the-Middle).
- 2. Đồng bộ mã nguồn an toàn (Fast-forward Synchronization):** Lệnh kéo mã nguồn sử

dùng cờ `--ff-only` để đảm bảo lịch sử git trên máy chủ luôn là một đường thẳng khớp chính xác với commit đã qua kiểm thử trên nhánh `main`:

```
cd /opt/library74
git pull --ff-only origin main
```

Cơ chế này ngăn chặn tuyệt đối tình trạng máy chủ tự động tạo merge commit bất thường ngoài tầm kiểm soát khi môi trường thực thi bị can thiệp.

3. **Kiểm tra tính hợp lệ của cấu hình triển khai:** Trước khi thay đổi bất kỳ trạng thái nào của dịch vụ đang chạy, Docker Compose thực hiện kiểm tra cú pháp và tham chiếu biến môi trường:

```
docker compose --env-file .env.prod \
  -f docker-compose.prod.yml config >/dev/null
```

4. **Bảo vệ dữ liệu tự động (Pre-deployment Database Snapshot):** Đối với phân hệ Backend CD, nhằm phòng tránh rủi ro migration thất bại gây hỏng dữ liệu, hệ thống tự động kích hoạt container sao lưu cơ sở dữ liệu:

```
docker compose --env-file .env.prod -f docker-compose.prod.yml \
  --profile backup run --rm postgres-backup
```

Tác vụ này tạo ra bản chụp cơ sở dữ liệu nhị phân nén dạng `library-YYYYMMDD-HHM MSS.dump` lưu trữ trong thư mục `backups/` trên máy chủ trước khi phiên bản backend mới khởi chạy và chạy migration.

5. **Cập nhật dịch vụ độc lập không gián đoạn (Zero Interruption):** Hệ thống áp dụng cơ chế triển khai theo từng container chuyên biệt, tránh việc restart toàn bộ hệ thống gây gián đoạn:

- Backend: `docker compose ... up -d --build backend`
- AI Service: `docker compose ... up -d --build ai-api ai-worker`
- Frontend: `docker compose ... up -d --build frontend`

Các dịch vụ hạ tầng nền tảng như PostgreSQL, Kafka, RabbitMQ, Caddy Reverse Proxy vẫn duy trì kết nối liên tục 100% mà không bị ngắt quãng.

6. **Kiểm tra sức khỏe dịch vụ sau triển khai (Post-deployment Health Check):** Để khẳng định container mới không chỉ “đang chạy” (running) mà đã thực sự sẵn sàng phục vụ yêu cầu của người dùng, script CD thực hiện vòng lặp thăm dò (polling) tối đa 36 lần, mỗi lần cách nhau 5 giây (thời gian chờ tối đa 3 phút):

- **Backend:** Truy vấn actuator health endpoint nội bộ:

```
docker compose exec -T backend \
  wget -qO- http://127.0.0.1:8080/actuator/health | grep -q UP
```

- **AI Service:** Truy vấn endpoint kiểm tra trạng thái của FastAPI:

```
docker compose exec -T ai-api \
  curl -fsS http://127.0.0.1:8001/health
```

- **Frontend:** Kiểm tra đồng thời phản hồi HTTP của Nginx nội bộ và domain công

khai:

```
docker compose exec -T frontend wget -qO- http://127.0.0.1/ >/dev/null  
&& curl -fsS https://library74.uk/ >/dev/null
```

Nếu sau 36 lần thăm dò mà dịch vụ không phản hồi mã trạng thái thành công, pipeline lập tức báo lỗi *Deployment Failed* và gửi thông báo cho đội ngũ quản trị để kịp thời xử lý.

1.4.5 Quản lý bảo mật và phân tách cấu hình bí mật

Hệ thống tuân thủ nguyên tắc đặc quyền tối thiểu (Principle of Least Privilege) bằng cách phân tách cấu hình thành hai lớp bảo mật:

- **Biến môi trường nghiệp vụ (.env.prod):** Được lưu trữ cố định trên máy chủ sản xuất với phân quyền nghiêm ngặt (chmod 600, chỉ user quản trị hệ thống mới có quyền đọc/ghi). Các khóa nhạy cảm bao gồm mật khẩu database, S3 secret key, API key PayOS và token kết nối không bao giờ được đưa vào mã nguồn hay lưu trên kho lưu trữ git.
- **Biến môi trường triển khai (GitHub Repository Secrets):** Chỉ chứa các thông số định danh cần thiết để phục vụ quá trình kết nối và vận hành tự động: DEPLOY_HOST, DEPLOY_PORT, DEPLOY_USER, DEPLOY_PATH, khóa bảo mật DEPLOY_SSH_KEY và chuỗi định danh DEPLOY_KNOWN_HOSTS.

1.4.6 Các thách thức kỹ thuật và giải pháp xử lý thực tế

Trong quá trình xây dựng và vận hành thực tế hệ thống CI/CD cho đồ án, nhóm đã phát hiện và xử lý ba bài toán kỹ thuật chuyên sâu:

1. Xử lý lệch múi giờ (Timezone Mismatch) trong Unit Test Backend:

- **Hiện tượng:** Bài kiểm thử đơn vị nghiệp vụ tính tiền phạt trễ hạn mượn sách (ReturnBookUseCase.returnSuccess_withOverdueFine_3Days) liên tục thất bại ngẫu nhiên trên GitHub Actions với kết quả: expected: 3000 but was: 4000.
- **Nguyên nhân:** Logic nghiệp vụ tính ngày quá hạn dựa trên múi giờ Việt Nam `LocalDate.now(ZoneId.of("Asia/Ho_Chi_Minh")) (GMT+7)`. Trong khi đó, tệp test sử dụng phương thức mặc định `LocalDate.now()` lấy giờ hệ thống. Máy ảo GitHub Actions Runner thiết lập mặc định theo giờ UTC (GMT+0). Khi pipeline chạy vào khung giờ đêm ở Việt Nam (ví dụ 18:30 UTC = 01:30 hôm sau ở Việt Nam), sự chênh lệch ngày giữa hai múi giờ dẫn đến số ngày quá hạn bị tính dư 1 ngày (4 ngày thay vì 3 ngày).
- **Giải pháp:** Khắc phục bằng cách chỉ định tường minh `ZoneId.of("Asia/Ho_Chi_Minh")` trong toàn bộ các lớp test mock thời gian, đảm bảo tính toán đồng nhất độc lập với múi giờ của máy chủ thực thi.

2. Chuẩn hóa cú pháp tương tác container trong Docker Compose:

- **Hiện tượng:** Ban đầu lệnh kiểm tra trạng thái sử dụng cú pháp trực tiếp của Docker Daemon như `docker exec lms_backend ...`. Lệnh này phát sinh lỗi do

Docker Compose tự động gán tiền tố thư mục dự án và hậu tố số thứ tự (ví dụ `library74-backend-1`), dẫn đến lỗi không tìm thấy container.

- *Giải pháp:* Chuyển sang sử dụng chuẩn cú pháp quản trị dịch vụ của Docker Compose:

```
docker compose --env-file .env.prod -f docker-compose.prod.yml \
    exec -T <service_name> <command>
```

Lệnh này giúp pipeline tương tác chính xác với container theo tên định danh dịch vụ (*service name*), hoàn toàn không phụ thuộc vào quy tắc đặt tên container vật lý của Docker daemon.

3. Xung đột phân giải địa chỉ Loopback (IPv6 vs IPv4) trên Alpine Linux:

- *Hiện tượng:* Kiểm tra dịch vụ Frontend container đóng gói trên nền Alpine Linux qua địa chỉ `http://localhost/` bị từ chối kết nối (`Connection refused`).
- *Nguyên nhân:* Thư viện mạng của Alpine Linux ưu tiên phân giải tên miền `localhost` sang địa chỉ IPv6 loopback (`::1`), trong khi Nginx cấu hình chỉ lắng nghe trên giao thức IPv4 (`0.0.0.0:80`).
- *Giải pháp:* Cấu hình rõ ràng địa chỉ IPv4 loopback `http://127.0.0.1/` trong kịch bản kiểm tra sức khỏe của pipeline, đảm bảo kết nối nội bộ thông suốt.

1.4.7 Đánh giá định lượng hiệu quả hệ thống

Việc đưa hệ thống CI/CD vào vận hành chính thức đã nâng cao rõ rệt độ tin cậy, an toàn và tốc độ phát hành của dự án. Bảng 1.3 so sánh định lượng các chỉ số vận hành then chốt trước và sau khi triển khai giải pháp CI/CD.

Bảng 1.3: Đánh giá hiệu quả vận hành trước và sau khi áp dụng CI/CD

Tiêu chí đánh giá	Quy trình thủ công (Trước)	Hệ thống CI/CD (Sau)
Thời gian phát hành	15–30 phút (đăng nhập SSH, gõ lệnh thủ công từng dịch vụ).	Dưới 3 phút (tự động hóa 100% từ khi push code).
Kiểm định chất lượng mã nguồn	Phụ thuộc sự cẩn thận của lập trình viên, dễ bỏ sót test.	100% tự động vượt qua test suite trước khi được phép triển khai.
An toàn dữ liệu cơ sở dữ liệu	Thực hiện thủ công, dễ bị lãng quên trước khi cập nhật.	Tự động chụp snapshot nhị phân trước mỗi lần deploy Backend.
Mức độ gián đoạn dịch vụ (Downtime)	Downtime kéo dài do máy chủ bị chiếm dụng tài nguyên để build image.	Tối thiểu hóa downtime nhờ cập nhật tách biệt theo container.
Phát hiện lỗi phát hành	Phát hiện muộn thông qua phản ánh của người dùng hoặc kiểm tra thủ công.	Hệ thống Polling Health Check tự động bắt lỗi ngay trong 3 phút đầu.

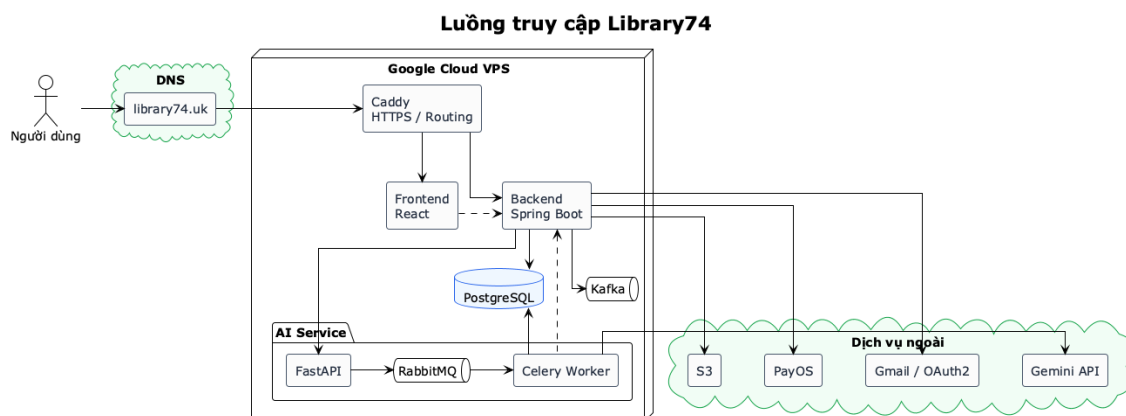
Kết quả cho thấy hệ thống CI/CD không chỉ loại bỏ hoàn toàn các thao tác thủ công rủi ro mà còn thiết lập một quy trình chuyển giao phần mềm tiêu chuẩn, an toàn và có khả năng phục hồi cao theo đúng chuẩn mực công nghiệp trong phát triển phần mềm hiện đại.

1.5 Hạ tầng mạng: Domain và HTTPS

1.5.1 Tên miền (Domain)

Trong bản triển khai demo, hệ thống sử dụng tên miền `library74.uk` để người dùng truy cập bằng một địa chỉ ổn định thay vì ghi nhớ IP hoặc cổng dịch vụ. DNS của domain được quản lý qua Cloudflare và trỏ về Google Cloud VPS. Tại VPS, Caddy nhận request HTTPS, tự động xử lý chứng chỉ TLS và định tuyến request đến frontend, backend API hoặc WebSocket theo path tương ứng.

Luồng kết nối sau khi đi vào backend tiếp tục được phân tách theo trách nhiệm: Spring Boot xử lý nghiệp vụ chính và kết nối PostgreSQL/Kafka; backend gọi AI Service qua `AI_GATEWAY_BASE_URL`; AI worker xử lý PDF bất đồng bộ qua RabbitMQ và callback kết quả về backend bằng chữ ký HMAC. Các tích hợp ngoài như S3-compatible storage, PayOS, Gmail SMTP, Google OAuth2 và Gemini API được kết nối từ backend hoặc AI worker theo đúng vai trò của từng dịch vụ.



Hình 1.2: Luồng kết nối từ tên miền Library74 đến hệ thống production

1.5.2 Tổng chi phí vận hành

Bảng 1.4 được điều chỉnh theo chi phí triển khai hiện tại. Tên miền `library74.uk` được mua theo năm, còn máy chủ demo đang chạy trên Google Cloud Platform với cấu hình ước tính khoảng 92 USD/năm. Tại thời điểm triển khai demo, GCP đang có ưu đãi miễn phí 3 tháng nên chi phí thực trả ban đầu thấp hơn chi phí vận hành ổn định về sau.

Như vậy, sau giai đoạn miễn phí ban đầu, chi phí cố định tối thiểu của hệ thống chủ yếu gồm tên miền và máy chủ GCP, khoảng 97,3 USD/năm theo cấu hình demo. Khi triển khai production thật, các thông số cần được cập nhật lại theo dung lượng PDF, số lượt truy vấn AI và chính sách giá của nhà cung cấp tại thời điểm vận hành.

Bảng 1.4: Chi phí vận hành hệ thống trong cấu hình demo

Thành phần	Chi phí	Chu kỳ	Ghi chú
Tên miền library74.uk	5,3 USD	Năm	Khoản bắt buộc để giữ địa chỉ public của demo.
SSL/HTTPS	Miễn phí	90 ngày hoặc tự động	Let's Encrypt hoặc chứng chỉ tự động từ reverse proxy/nhà cung cấp.
Google Cloud Platform	92 USD	Năm	Chi phí theo cấu hình máy chủ demo; hiện đang được miễn phí 3 tháng đầu.
Lưu trữ file	Theo mức sử dụng	Tháng	Ảnh bìa/PDF dùng object storage hoặc cấu hình tương đương; chi phí phụ thuộc dung lượng thực tế.
API ngoài	Miễn phí trong phạm vi demo	Tháng	Google OAuth, Gmail SMTP, PayOS sandbox/quota demo và Gemini quota thử nghiệm.
Tổng cố định ước tính	97,3 USD	Năm	Chưa tính ưu đãi miễn phí 3 tháng GCP và chi phí phát sinh theo dung lượng/request thực tế.

Chương 2: Tổng kết

Dự án “Hệ thống Quản lý Thư viện Trực tuyến tích hợp AI” (Library74) được định hình và phát triển với tầm nhìn chuyển đổi số quy trình vận hành thư viện. Vượt qua giới hạn của một phần mềm quản lý thông thường, dự án đã hiện thực hóa một kiến trúc phân tán bao gồm ba phân lớp lõi: Frontend, Backend và AI Service.



Hình 2.1: Nhận diện thương hiệu của nền tảng Library74

2.1 Tổng hợp kết quả hiện thực

Đóng góp cốt lõi của đồ án không nằm ở việc xây dựng các module đơn lẻ, mà ở khả năng liên kết các phân hệ thành những luồng dữ liệu và luồng nghiệp vụ khép kín, hoạt động ổn định trên môi trường triển khai thực tế.

2.1.1 Phân lớp Frontend

Thay vì liệt kê các màn hình rời rạc, lớp giao diện được định hình theo nhóm người dùng để tối ưu hóa trải nghiệm tương tác:

- **Trang công cộng:** Triển khai cơ chế kết xuất trang tĩnh và kết xuất phía máy chủ cho trang chủ, luồng tìm kiếm, đăng nhập OAuth2 và khôi phục mật khẩu. Điều này giúp giảm thiểu rào cản tiếp cận, tối ưu hóa công cụ tìm kiếm (SEO) và cung cấp điểm chạm đầu tiên mượt mà cho độc giả.
- **Khu vực độc giả:** Xây dựng dashboard cá nhân hóa, quản lý vòng đời phiếu mượn, thanh toán mã QR, hệ thống Wishlist, Rating và thông báo thời gian thực. Các tính năng này giao quyền chủ động cho độc giả trong việc tự phục vụ và theo dõi tài nguyên học tập cá nhân.
- **Khu vực vận hành:** Thiết kế giao diện quản lý vòng đời ấn phẩm, kiểm soát hàng chờ, xử lý phạt và mất sách cùng dashboard điều hành. Phân hệ này hỗ trợ số hóa nghiệp vụ thủ công, gom cụm các thao tác vận hành vào một không gian làm việc duy nhất.
- **Khu vực quản trị:** Cung cấp công cụ cấu hình chính sách thư viện, quản trị định danh, phân quyền và giám sát hệ thống qua Audit Log, đảm bảo tính toàn vẹn, bảo mật và khả

năng kiểm toán hệ thống minh bạch.

- **Tối ưu nền tảng:** Áp dụng quản lý trạng thái linh hoạt, hỗ trợ đa ngôn ngữ, hệ thống giao diện sáng tối, Route Guard và luồng Refresh Token ngầm để duy trì sự liền mạch trong các phiên làm việc kéo dài.

2.1.2 Phân lớp Backend

Máy chủ trung tâm đảm nhiệm vai trò kiểm soát toàn vẹn dữ liệu và điều phối các luồng xử lý phức tạp:

- **Quản trị định danh:** Sử dụng JWT, cơ chế xoay vòng token, xác thực email, Google OAuth2 và kiểm soát quyền truy cập dựa trên vai trò để tạo lập rào chắn bảo mật vững chắc cho toàn bộ API.
- **Quản lý tài nguyên:** Chuẩn hóa lược đồ cho ấn phẩm, tác giả, danh mục và tích hợp Storage Port lưu trữ file số an toàn, định hình nguồn dữ liệu chuẩn xác cho tài nguyên thư viện.
- **Luồng mượn trả:** Xử lý giao dịch mượn trả tuân thủ nguyên tắc ACID, quản lý hàng chờ đặt trước, xử lý ngoại lệ trễ hạn và mất sách, giải quyết bài toán cốt lõi của nghiệp vụ thư viện truyền thống.
- **Thanh toán và sự kiện:** Tích hợp cổng PayOS qua Webhook, xây dựng kiến trúc hướng sự kiện với Kafka cho thông báo và Scheduler định kỳ nhắc hạn nhằm thúc đẩy tự động hóa và giảm độ trễ phản hồi.
- **Hạ tầng và hệ thống:** Quản lý lược đồ dữ liệu với Flyway, xử lý lỗi tập trung và tích hợp AI Gateway với cơ chế bảo mật chữ ký HMAC, giúp hệ thống dễ dàng mở rộng và bảo trì trong dài hạn.

2.1.3 Phân lớp AI Service

Phân hệ trí tuệ nhân tạo được thiết kế như một dịch vụ độc lập nhằm tránh gây áp lực lên luồng xử lý đồng bộ của hệ thống chính:

- **Luồng xử lý dữ liệu PDF:** Tự động trích xuất ngữ nghĩa bằng PyMuPDF, làm sạch nhiễu, phân mảnh văn bản và tối ưu tính toán để bỏ qua các file trùng lặp, giúp chuyển đổi dữ liệu phi cấu trúc thành định dạng tiêu chuẩn.
- **Mô hình nhúng vector:** Khởi tạo vector đa chiều với SentenceTransformer chuẩn hóa tiếng Việt, lưu trữ và truy xuất thông qua pgvector trên PostgreSQL để giải quyết bài toán tìm kiếm ngữ nghĩa theo khoảng cách.
- **Làm giàu siêu dữ liệu:** Điều phối mô hình ngôn ngữ sinh tóm tắt, trích xuất nhãn đa ngữ và giới hạn tình trạng ảo giác qua cơ chế ép định dạng JSON, giúp giảm thiểu đáng kể công việc nhập liệu thủ công của thủ thư.
- **Hệ thống gợi ý:** Áp dụng thuật toán kết hợp giữa lọc cộng tác, phân tích nội dung và cơ

chế đề xuất theo xu hướng cho người dùng mới, từng bước cá nhân hóa trải nghiệm khám phá tri thức.

- **Kiến trúc bất đồng bộ:** Tách biệt xử lý bằng FastAPI và Celery Worker qua RabbitMQ, giao tiếp an toàn với Backend qua mã hóa HMAC để ngăn chặn hiện tượng nghẽn cổ chai khi xử lý tác vụ AI nặng.

2.1.4 Kiểm thử và Đảm bảo chất lượng

Để minh chứng cho tính ổn định, hệ thống đã trải qua các kịch bản kiểm thử phân tầng với kết quả khả quan:

- **Kiểm thử Frontend:** Vượt qua 62/62 kịch bản Unit Test và Component Test cho logic giao diện, quản lý trạng thái và tính hợp lệ của biểu mẫu, đảm bảo tính chống chịu lỗi phía người dùng.
- **Kiểm thử Backend:** Vượt qua 79/79 kịch bản Integration Test tập trung vào logic nghiệp vụ, luồng giao dịch cơ sở dữ liệu và bảo mật API. Luồng cập nhật dữ liệu hoạt động nhất quán, không xảy ra hiện tượng bế tắc.
- **Đánh giá AI Service:** Hoàn tất 72/72 Contract Test cho luồng xử lý tự động. Việc đo lường trên tập dữ liệu chuẩn ghi nhận nDCG@5 đạt 0.907, Recall@5 đạt 87.5% và độ trễ p95 ở mức 540ms.
- **Vận hành thực tế:** Hệ thống vượt qua 8/8 kịch bản Smoke Test trên tên miền thật. Ổ bài toán thăm dò chịu tải với tối đa 200 người dùng ảo đồng thời, hệ thống duy trì tỷ lệ lỗi 0.00% và đảm bảo phản hồi ổn định.

2.1.5 Đóng gói và Triển khai hạ tầng

Hệ thống được đưa lên môi trường mạng Internet với kiến trúc triển khai tiêu chuẩn:

- **Nền tảng và môi trường:** Toàn bộ hệ sinh thái được đóng gói container và điều phối bởi Docker Compose trên nền tảng Google Cloud VPS sử dụng Ubuntu 24.04.
- **Phân giải tên miền và bảo mật:** Vận hành chính thức tại tên miền `https://library74.uk`, định tuyến DNS qua mạng lưới Cloudflare. Máy chủ Caddy làm định tuyến ngược, tự động cấp phát chứng chỉ SSL để mã hóa giao tiếp.
- **Vận hành Frontend và Backend:** Ứng dụng React được biên dịch tĩnh và phân phối qua máy chủ Nginx. Khối Spring Boot liên kết chặt chẽ với các dịch vụ nội bộ (PostgreSQL, Kafka, RabbitMQ) và nền tảng bên ngoài (S3, PayOS, Google).
- **Vận hành AI Engine và dữ liệu:** Môi trường FastAPI vận hành độc lập cùng hệ thống Worker lắng nghe hàng đợi. Hệ quản trị PostgreSQL được ánh xạ bộ nhớ độc lập, trong khi hình ảnh và PDF được đồng bộ với Amazon S3 để dự phòng.

2.2 Đánh giá tác động thực tiễn

Dự án Library74 đã chứng minh được tính khả thi trong việc ứng dụng nền tảng công nghệ để giải quyết bài toán vận hành thực tế của thư viện đại học. Tác động của hệ thống không mang tính lý thuyết mà mang lại giá trị vận hành cụ thể cho từng nhóm đối tượng tham gia:

- **Đối với sinh viên và giảng viên:** Dưới sự hỗ trợ của công cụ tìm kiếm ngữ nghĩa và hệ thống gợi ý, khoảng cách từ nhu cầu tra cứu học liệu đến quyết định mượn sách được thu hẹp đáng kể. Nền tảng cung cấp một luồng tự phục vụ xuyên suốt, giúp bạn đọc chủ động và nắm toàn quyền kiểm soát quá trình mượn trả tài liệu của cá nhân.
- **Đối với thủ thư:** Hệ thống giải phóng nhân sự khỏi khối lượng lớn các công việc hành chính thủ công. Thông qua các tính năng quản lý vòng đời tài liệu, đối soát thanh toán tự động và xử lý hàng chờ trực tuyến, các bước nghiệp vụ được chuẩn hóa. Điều này giúp hạn chế tối đa sai sót do thao tác con người, từ đó nâng cao hiệu suất phục vụ tổng thể.
- **Đối với nhà trường:** Hệ thống kiến tạo một kho dữ liệu tập trung, lưu vết toàn bộ vòng đời tương tác của tài nguyên học liệu. Đây là nguồn dữ liệu nền tảng thiết yếu để cấp quản lý phân tích xu hướng tìm kiếm, đánh giá mức độ lưu thông sách và đưa ra các quyết định hoạch định chiến lược đầu tư tài nguyên mới một cách có cơ sở.
- **Đối với bài toán chuyển đổi số:** Dự án khẳng định quá trình chuyển đổi số thư viện không đơn thuần dừng lại ở việc chuyển đổi file tài liệu lên máy chủ. Bản chất cốt lõi của giải pháp là sự số hóa toàn diện luồng tương tác giữa con người và tri thức, kết hợp linh hoạt các công cụ học máy để tạo ra bước tiến thực sự về chất lượng dịch vụ.

2.3 Nhận định kiến trúc và giới hạn kỹ thuật

Dựa trên kết quả thực nghiệm, dự án ghi nhận các ưu điểm trong thiết kế kiến trúc cũng như các rào cản kỹ thuật cần khắc phục để đưa hệ thống lên mức chịu tải quy mô lớn hơn:

- **Kiến trúc nghiệp vụ:** Đạt được sự ổn định nhờ tính toàn vẹn cho các giao dịch mượn trả và phân rã module rõ ràng. Tuy nhiên, do chưa áp dụng kiến trúc vi dịch vụ hoàn chỉnh, hệ thống có thể đối mặt với rào cản mở rộng cục bộ nếu một module gặp tình trạng quá tải đột biến.
- **Trí tuệ nhân tạo:** Kiến trúc tách rời giúp luồng backend không bị ảnh hưởng bởi quá trình suy luận của AI. Dù vậy, hệ thống gợi ý vẫn chịu ảnh hưởng của hiện tượng khởi động nguội do thiếu tập dữ liệu tương tác thực, đồng thời siêu dữ liệu dễ bị nhiễu nếu file PDF đầu vào có định dạng sai lệch.
- **Hạ tầng và vận hành:** Hệ thống dễ dàng tái tạo tự động thông qua cấu trúc mã hóa và lớp bảo mật định tuyến nghiêm ngặt. Điểm hạn chế là việc kiểm thử tải mới dừng ở mức thăm dò có kiểm soát, chưa thực hiện các kịch bản kiểm thử giới hạn cực đại để xác định điểm gãy của phần cứng.

2.4 Định hướng phát triển

Để nâng tầm Library74 thành một nền tảng phục vụ cho các đại học quy mô lớn, các trọng tâm kỹ thuật trong giai đoạn tiếp theo bao gồm:

- **Xây dựng hệ thống giám sát toàn diện:** Tích hợp các công cụ theo dõi đo lường và ghi log theo thời gian thực, đồng thời cài đặt hệ thống cảnh báo tự động để phát hiện nguy cơ trước khi máy chủ chạm ngưỡng thất cố chai.
- **Khắc phục nhiễu dữ liệu đầu vào:** Bổ sung công nghệ nhận dạng ký tự quang học kết hợp thuật toán phân tích bố cục hình ảnh để trích xuất văn bản từ sách scan cổ, sách mờ hoặc có thiết kế dàn trang phức tạp.
- **Tối ưu hóa hệ thống gợi ý:** Chuyển dịch dần từ việc phân tích dữ liệu tĩnh sang nắm bắt hành vi ngầm của độc giả, chẳng hạn như thời gian dừng trên trang hoặc tỷ lệ nhấp chuột, nhằm tinh chỉnh mô hình đề xuất sát với thực tế hơn.
- **Cơ chế dự phòng và phục hồi:** Tự động hóa hoàn toàn luồng sao lưu gia tăng cho cơ sở dữ liệu và thiết lập các kịch bản chuyển đổi dự phòng để đảm bảo tính sẵn sàng cao cho hệ thống vận hành.

2.5 Kết luận chung

Đồ án "**Xây dựng Hệ thống Quản lý Thư viện Trực tuyến tích hợp AI**" đã hoàn thành các mục tiêu nghiên cứu và kỹ thuật được đặt ra ban đầu. Dự án không chỉ dừng lại ở mức độ nghiên cứu lý thuyết mà đã phát triển thành một hệ thống phần mềm có mức độ hoàn thiện ổn định, đáp ứng được các yêu cầu vận hành cơ bản.

Bằng việc kết hợp hài hòa giữa kiến trúc phần mềm, quy trình nghiệp vụ thư viện và khả năng hỗ trợ của các thuật toán học máy, Library74 là một ví dụ cụ thể cho việc áp dụng nền tảng kỹ thuật công nghệ vào kiến tạo những giải pháp mang lại giá trị thực tiễn cho môi trường giáo dục đại học.

Tài liệu tham khảo

- [1] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-Networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 3982–3992.
- [2] Y. Hu, Y. Koren, and C. Volinsky, “Collaborative filtering for implicit feedback datasets,” in *8th IEEE International Conference on Data Mining (ICDM 2008)*. IEEE, 2008, pp. 263–272.
- [3] R. Burke, “Hybrid recommender systems: Survey and experiments,” *User Modeling and User-Adapted Interaction*, vol. 12, no. 4, pp. 331–370, 2002.
- [4] A. Kane, *pgvector: Open-source vector similarity search for PostgreSQL*, 2024, accessed: 2026-05-01. [Online]. Available: <https://github.com/pgvector/pgvector>
- [5] F5, Inc., “NGINX documentation,” <https://nginx.org/en/docs/>, 2024, accessed: 2026-05-01.

Tài nguyên và Mã nguồn dự án

Để phục vụ cho quá trình thiết kế, kiểm thử và đánh giá, báo cáo đính kèm các đường dẫn đến tài nguyên của dự án:

Loại tài nguyên	Đường dẫn truy cập
Mã nguồn (Source Code)	Frontend: https://github.com/imsythang/LMS_fe Backend: https://github.com/quangthangk4/LMS_BE AI: https://github.com/howard-hosythang/LMS_AI
Website triển khai	https://library74.uk
Thiết kế giao diện (Figma)	https://www.figma.com/design/aSHkyebIWfWN7LMgTeY0be/Library-Management-System?node-id=0-1&t=mf5nzMKPeV3M6Vyp-1

Bảng 1: Danh sách các tài nguyên trực tuyến của hệ thống